INDIVIDUAL PROJECT REPORT

Review 2 — Design, Development & Progress (14/9/2026)

AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry

Student	Register No.	Team No.	My role
Anvay Sharma	24BCE0824	_____	Edge gateway and edge intelligence

Team: Laksh Ahuja (24BDS0212), Anvay Sharma (24BCE0824), Sreehari (24BDS0417). **Repository:** https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK

Summary

Since Review 1 I built the edge gateway of our prototype: it receives every sensor packet, classifies it as CRITICAL or NORMAL against cyclone thresholds, forwards critical packets first through a priority queue, and throttles normal traffic while a cyclone alert is active. This report covers the gateway's design, the working code, my design choices, feasibility, the problems I found by testing it, and my contribution.

1. Detailed Design

The gateway has two threads (Figure 1). The **receiver** reads UDP port 9001, decodes the record, classifies it and pushes (priority, order, record) into a priority queue. The **forwarder** takes the highest-priority record and sends it to the server on port 9002; while an alert is active, each NORMAL packet waits 0.5 s first. The alert switches on with any CRITICAL packet and clears after 10 s without one.

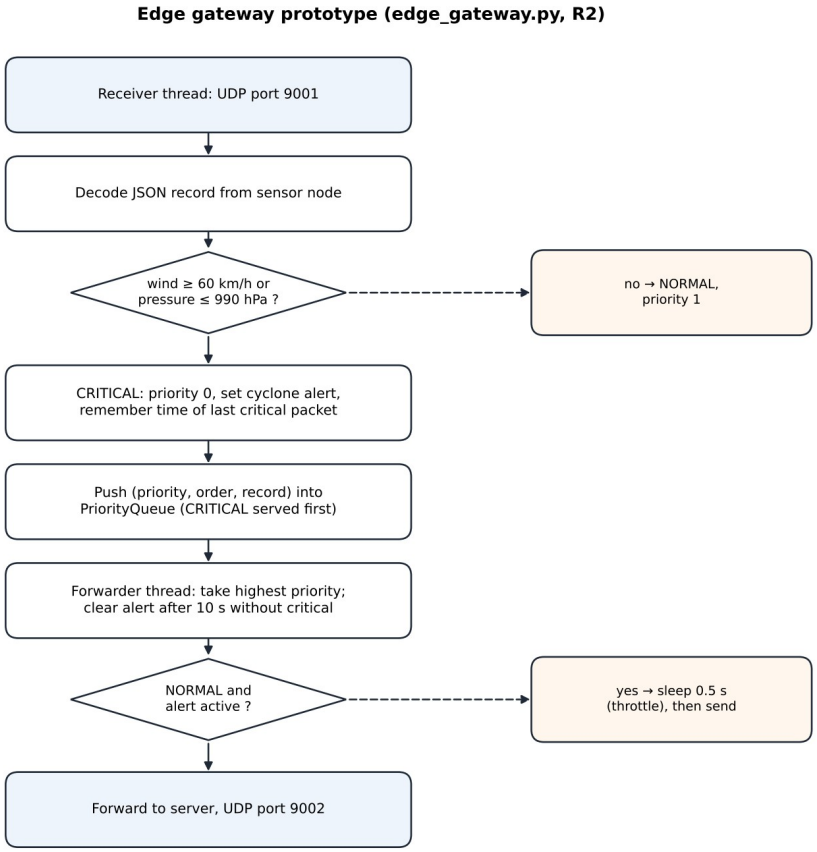


Figure 1: Flowchart of my edge gateway (prototype)

Table 1: Labelled parameters of my module

Parameter	Value	Meaning
SENSOR_TO_GATEWAY_PORT	9001 (UDP)	Input from sensor nodes
GATEWAY_TO_SERVER_PORT	9002 (UDP)	Output to the central server
WIND_CRITICAL_THRESHOLD	60 km/h	CRITICAL if wind ≥ this
PRESSURE_CRITICAL_THRESHOLD	990 hPa	CRITICAL if pressure ≤ this
Priority	0 = CRITICAL, 1 = NORMAL	Lower value served first; insertion order breaks ties
ALERT_HOLD_SECONDS	10 s	Alert stays on this long after the last CRITICAL packet
Throttle	0.5 s per NORMAL packet	Applied only while the alert is active

2. Actual Working Progress since Review 1

- **Code committed:** `edge_gateway.py` (101 lines) and the `classify_traffic()` rule in `common.py`, commit `82bcb8a`.
- **Running:** the gateway receives from three nodes and forwards in priority order; the log shows the

alert switching on and NORMAL packets being throttled.

- **Studied for the next phase:** trend-based ML classification and satellite feature extraction (literature from Review 1).

```
def classify_traffic(record):
    if record["wind"] >= WIND_CRITICAL_THRESHOLD or record["pressure"] <=
PRESSURE_CRITICAL_THRESHOLD:
        return TRAFFIC_CRITICAL
    return TRAFFIC_NORMAL

# forwarder thread
_priority, _order, record = pq.get()
if record["traffic_class"] != TRAFFIC_CRITICAL and cyclone_alert.is_set():
    time.sleep(0.5)            # reserve bandwidth for critical traffic
sock.sendto(encode(record), (HOST, GATEWAY_TO_SERVER_PORT))
```

To produce dated evidence for this report, the Review 2 prototype (repository commit `82bcb8a`, 16/9/2026) was re-run unchanged with three sensor nodes for 60 seconds; the logs are in `docs/reports/evidence/r2_prototype/`. The team's original 12-second test reported 66 packets received, 11 reconstructed and 2 critical, with overall risk 100 and the alert active.

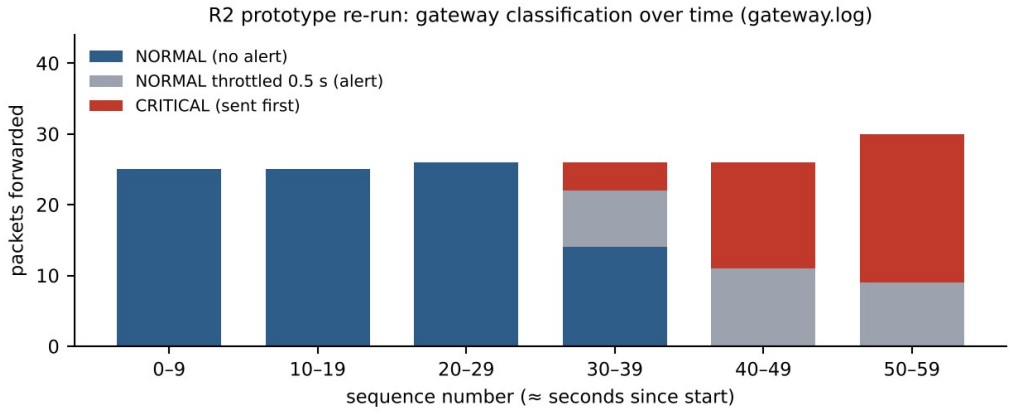


Figure 2: Classification over time in the re-run (from my gateway's log)

In the re-run the gateway forwarded 160 packets: 42 CRITICAL packets were sent immediately, and 28 NORMAL packets were throttled during the alert.

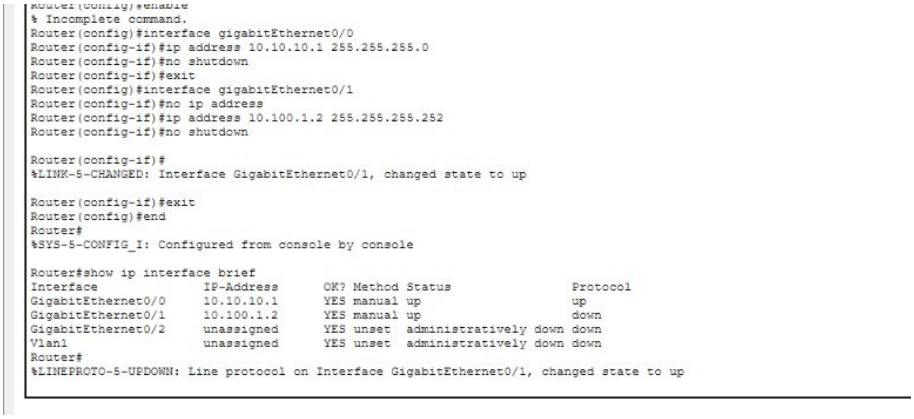


Figure 3: Ocean-side router of the team's Packet Tracer topology (joint work); this is the edge of the network where my gateway sits

Completion: about 40 % of my planned scope. **Done:** gateway pipeline, threshold classification, priority forwarding, alert mode. **Remaining:** input validation, an ML early-warning classifier, satellite image feature extraction, an LSTM for missing data, and a proper evaluation.

Table 2: Test runs performed on my module

Test run	Setup	Observation
Team test, 12 s, 2 nodes	Gateway between sensors and server	2 packets classified CRITICAL once wind > 60

Test run	Setup	Observation
Re-run, 60 s, 3 nodes	Prototype commit 82bcb8a	160 forwarded; 42 CRITICAL sent first; 28 NORMAL throttled
Robustness test	One non-JSON datagram to port 9001	Receiver thread died; later packets ignored
Alert-hold check	Code review of the forwarder loop	Alert can be stale after a silent period

3. Justification of Design Choices

Table 3: Comparison of alternatives for my module

Choice	Option	Accuracy / quality	Compute & power	Ease	Decision
Classifier	**Threshold rule**	Exact on breaches; no early warning	Negligible	Very easy	Chosen now
	Logistic regression on trends	Can warn before breach	Very low	Easy	Next phase
	Deep network	Possibly best	High; needs GPU/data	Hard	Rejected
Forwarding	FIFO queue	Critical waits behind others	Low	Easiest	Rejected
	Fixed bandwidth cap per class	Wastes capacity when idle	Low	Easy	Rejected
	Priority queue	Critical first, adapts instantly	Low	Easy	Chosen
Language	**Python threads**	Fast enough at our rates	Runs on laptop / Pi	Easy	Chosen

4. Updated Feasibility

Table 4: Economic picture for my part

Aspect	Review 1 estimate	Now (Review 2)	Reason
Software cost	₹0	₹0	Python standard library only
Edge hardware	Laptop or Raspberry Pi	Laptop; Pi optional (lab)	Gateway uses < 1 % CPU
ML compute	Possibly GPU	CPU only	Lightweight models chosen to fit the edge

Social: earlier recognition of danger directly benefits people who must evacuate. **Environmental:** choosing small CPU models instead of GPU training keeps energy use low, and prioritising data avoids wasteful retransmissions on remote links.

5. Problems Encountered and How I Adapted

Table 5: Problems in my module

Problem	How I diagnosed it	What I changed	Result
A malformed packet stopped the gateway	Sent a non-JSON datagram while testing: the receiver thread raised an exception in decode() and died; later valid packets were ignored	Added a validation step to the design (check fields and ranges, count and drop bad packets)	To be implemented next; logged as the top fix
NORMAL packets delayed even on an idle link	Log showed 28 NORMAL packets each waiting 0.5 s during the alert although the queue was empty; a fixed sleep ignores real link	Plan a capacity-based scheduler (token bucket) instead of a fixed sleep	Behaviour understood; redesign scheduled

Problem	How I diagnosed it	What I changed	Result
	capacity		
Alert state can be stale	Code review: the alert is cleared only before the forwarder blocks on the queue, so a NORMAL packet arriving after a long silence is still throttled	Re-check the alert after every dequeue in the redesign	Identified before it affected a demo

Plan from Review 2 to Review 3

Week	Planned work
15–21 Sep	Input validation and capacity-based forwarding in the gateway
22–28 Sep	Trend-based ML classifier (logistic regression) with the rule as safety floor
29 Sep–5 Oct	Satellite feature extraction; LSTM implementation in NumPy
6–9 Oct	Evaluation harness, results tables, trained models in the repository

6. Individual Contribution Evidence

Table 6: Division of work

Tasks I did	Tasks done by teammates
Edge gateway: receiver and forwarder threads, priority queue	Laksh: central server, REST API, dashboard, startup fix
CRITICAL / NORMAL rule (classify_traffic) and alert mode	Sreehari: sensor-node simulator and gap interpolation
Robustness testing of the gateway (malformed input, throttling)	Team: shared record format, Packet Tracer topology

- Code: `edge_gateway.py` and `classify_traffic()` in `common.py`, commit `82bcb8a` (https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK).
- Logs: `gateway.log` and `crash_gw.log` in `docs/reports/evidence/r2_prototype/`.

> *[Insert here: screenshot of my own commits / work log and a photo or screen recording of me testing. Note: the team's early commits were pushed from one shared GitHub account, so the file-level split above is the record of who wrote what.]*

R2 Video Plan

Segment	What I show
Design walkthrough (2 min)	Face + screen; flowchart and parameter table
Live progress with my own work on screen (2 min)	Face + screen; code running and the logs
One problem and its solution (1 min)	Face + screen; the problem table and the evidence log
Updated cost and my contribution (1 min)	Face + screen; cost table and the division-of-work table